

CODE:

```
from dronekit import connect, VehicleMode, LocationGlobalRelative
import time

# -----
# Connect to vehicle
# -----
# Direct connection - SITL is the ONLY thing this script talks to.
# No Mission Planner, no MAVProxy. Just: start SITL, then run this script.
connection_string = 'tcp:127.0.0.1:5760'
print(f"Connecting to vehicle on: {connection_string}")
vehicle = connect(connection_string, wait_ready=True)

# -----
# Helper: Arm and Takeoff
# -----
def arm_and_takeoff(aTargetAltitude):
    print("Basic pre-arm checks...")
    while not vehicle.is_armable:
        print(" Waiting for vehicle to initialise...")
        time.sleep(1)

    print("Arming motors")
    vehicle.mode = VehicleMode("GUIDED")
    vehicle.armed = True

    while not vehicle.armed:
        print(" Waiting for arming...")
        time.sleep(1)

    print(f"Taking off to {aTargetAltitude}m")
    vehicle.simple_takeoff(aTargetAltitude)

    while True:
        current_alt = vehicle.location.global_relative_frame.alt
        print(f" Altitude: {current_alt}")
        if current_alt >= aTargetAltitude * 0.95:
            print("Reached target altitude")
```

```
        break
    time.sleep(1)
```

```
# -----
```

```
# Helper: Go to a waypoint
```

```
# -----
```

```
def goto_point(lat, lon, alt, label="Waypoint"):
    print(f"Heading to {label}: ({lat}, {lon}, {alt}m)")
    point = LocationGlobalRelative(lat, lon, alt)
    vehicle.simple_goto(point)
    time.sleep(15) # allow time to travel - adjust based on distance
```

```
# -----
```

```
# Helper: Return home and land
```

```
# -----
```

```
def return_and_land():
    print("Returning to Launch (RTL)")
    vehicle.mode = VehicleMode("RTL")
    time.sleep(5)
```

```
# =====
```

```
# MISSION 1: SQUARE PATH (Deliverable 4)
```

```
# =====
```

```
def square_mission():
    print("\n--- Starting SQUARE Mission ---")
    arm_and_takeoff(10)
```

```
    # Base coordinates (replace with your SITL home location)
```

```
    base_lat = -35.363261
```

```
    base_lon = 149.165230
```

```
    offset = 0.0005 # ~50m approx per step
```

```
    goto_point(base_lat, base_lon, 10, "Point A")
```

```
    goto_point(base_lat + offset, base_lon, 10, "Point B")
```

```
    goto_point(base_lat + offset, base_lon + offset, 10, "Point C")
```

```
    goto_point(base_lat, base_lon + offset, 10, "Point D")
```

```
return_and_land()
print("--- Square Mission Complete ---\n")
```

```
# =====
# MISSION 2: TRIANGLE PATH (Deliverable 4)
# =====
```

```
def triangle_mission():
    print("\n--- Starting TRIANGLE Mission ---")
    arm_and_takeoff(10)
```

```
base_lat = -35.363261
base_lon = 149.165230
offset = 0.0005
```

```
goto_point(base_lat, base_lon, 10, "Point A")
goto_point(base_lat + offset, base_lon + offset, 10, "Point B")
goto_point(base_lat, base_lon + (2 * offset), 10, "Point C")
```

```
return_and_land()
print("--- Triangle Mission Complete ---\n")
```

```
# =====
# MISSION 3: AUTONOMOUS PATROL MISSION (Deliverable 5)
# 4 Waypoints -> Maintain Altitude -> Return Home -> Land
# =====
```

```
def patrol_mission():
    print("\n--- Starting Autonomous Patrol Mission ---")
    target_altitude = 15
    arm_and_takeoff(target_altitude)
```

```
base_lat = -35.363261
base_lon = 149.165230
offset = 0.0006
```

```
waypoints = [
    (base_lat + offset, base_lon, target_altitude, "Patrol Point 1"),
    (base_lat + offset, base_lon + offset, target_altitude, "Patrol Point
2"),
```

```
(base_lat, base_lon + offset, target_altitude, "Patrol Point 3"),  
(base_lat - offset, base_lon, target_altitude, "Patrol Point 4"),  
]
```

```
for lat, lon, alt, label in waypoints:  
    goto_point(lat, lon, alt, label)  
    # Altitude check at each point  
    current_alt = vehicle.location.global_relative_frame.alt  
    print(f" Maintained altitude at {label}: {current_alt:.2f}m")
```

```
return_and_land()  
print("--- Patrol Mission Complete ---\n")
```

```
# =====  
# RUN A MISSION (uncomment the one you want to test)  
# =====  
if __name__ == "__main__":  
    # square_mission()  
    # triangle_mission()  
    patrol_mission()  
  
print("Closing vehicle connection")  
vehicle.close()
```

OUTPUT:

```
Python 3.10.8 (tags/v3.10.8:aaaf517, Oct 11 2022, 16:50:30) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/dell/waypoint.py =====
Connecting to vehicle on: tcp:127.0.0.1:5760
CRITICAL:autopilot:APM:Copter V3.3 (d6053245)
CRITICAL:autopilot:Frame: QUAD
CRITICAL:autopilot:Calibrating barometer
CRITICAL:autopilot:Initialising APM...
CRITICAL:autopilot:barometer calibration complete
CRITICAL:autopilot:GROUND START

--- Starting Autonomous Patrol Mission ---
Basic pre-arm checks...
Waiting for vehicle to initialise...
Waiting for vehicle to initialise...
Waiting for vehicle to initialise...
Waiting for vehicle to initialise...
Waiting for vehicle to initialise...
Waiting for vehicle to initialise...
Arming motors
Waiting for arming...
Waiting for arming...
Waiting for arming...
Waiting for arming...ERROR:autopilot:ARMING MOTORS

CRITICAL:autopilot:GROUND START
WARNING:dronekit:Link timeout, no heartbeat in last 5 seconds
Waiting for arming...
CRITICAL:autopilot:Initialising APM...
Taking off to 15m
Altitude: 0.0
```